

Предварительное многоролевое планирование для программирования и vibe coding

Халилов Джабраиль Эльнурович

Национальный исследовательский университет «Высшая школа экономики»

Исследовательский проект Xynapse / BVC

github.com/jabrailkhalil/xynapse

2026

Аннотация

В интерактивном программировании пользователь часто понимает необходимость дополнительной проверки до изменения кода: например, когда план составлен несколькими агентами, дефект затрагивает несколько компонентов или цена ошибочного первого решения высока. В статье исследуется upfront-сценарий Bounded Verification Council (BVC), который человек явно включает до первой правки. Его практическая сила состоит в понятном контракте: четыре роли независимо анализируют причину, стратегию, зависимости и тесты; формализованный gate либо немедленно синтезирует план, либо добавляет один ограниченный раунд критики. На 60 реальных задачах SWE-bench Verified BVC решил 5 задач против 7 у одного предварительного плана: первичная разность $-3,3$ процентного пункта, 95% интервал $[-10,0; +3,3]$, $p = 0,625$; преимущество и заранее заданная неухудшаемость с margin -5 п. п. не установлены. Во вторичном сравнении BVC решил 5 задач против 4 у прямой генерации patch: описательная разность $+1,7$ процентного пункта, 95% стратифицированный bootstrap-интервал $[0,0; +5,0]$, точный и Holm-adjusted $p = 1,0$. Это положительный сигнал, но не статистическое доказательство преимущества качества. В контролируемой 15-задачной разговорной серии BVC решил 1 задачу против 0 в формальной постановке и сохранил одинаковый outcome на 14/15 задачах. Одновременно gate завершил 32/60 задач коротким маршрутом и сократил токены на 22,2% относительно полного совета. Результаты показывают, как предложенный в дипломе интерфейс Xynapse можно превратить в полезный и прозрачный режим «сначала проверить план, затем писать код» с измеримым бюджетом и честной границей статистических утверждений.

Ключевые слова: vibe coding, AI-ассистент, предварительное планирование, многоагентный анализ, исправление дефектов, Xynapse, BVC, SWE-bench Verified.

1. Введение

Разговорное программирование снижает стоимость формулировки задачи: человек может описать желаемое поведение обычным языком и сразу перейти к генерации изменений. Слабое место такого взаимодействия проявляется до первой правки. Если исходная локализация дефекта неверна, последующий инструментальный цикл может уверенно редактировать неподходящий файл, расширять поверхность patch и исправлять симптомы вместо причины.

В настоящей работе термин *vibe coding* операционализирован узко: это одна созданная моделью разговорная переформулировка исходного требования при неизменных кодовом контексте, модели, лимитах и evaluator, без последующего диалога. Интерактивное программирование с уточнениями пользователя, инструментами и несколькими циклами правки здесь не исследуется.

Практический ответ пользователя нередко звучит так: «сначала составь план с агентами» или «перед исправлением ошибки проверь решение с нескольких сторон». Это не требование бесконечного автономного роя. Скорее, пользователь хочет короткий этап принятия инженерного решения, отделённый от редактирования кода.

В дипломе автора Xynapse описан как интеллектуальная среда разработки с интегрированным AI-ассистентом, а BVC — как механизм многоролевой проверки до действия [10]. Настоящая работа проверяет

этот сценарий на реальных программных задачах. Центральное свойство называется *upfront activation*: алгоритм включается человеком до первой правки, формирует единый план, а уже затем передаёт его patch-генератору. Актуальная реализация Хунапсе опубликована в репозитории проекта [9]; этим самым ссылка на программный код отделяется от ссылки на описание архитектуры в дипломе.

Исследование не сводится к сравнению красивых планов. Итоговый patch оценивается официальными тестами SWE-bench Verified [2]; одни и те же задачи попарно проходят direct, один план и многоролевые режимы. Дополнительная серия проверяет контролируруемую разговорную формулировку.

Что проверено

- момент включения: BVC запускается до первой правки;
- реальный outcome: применимость patch и прохождение тестов;
- первичный contrast с одним планом и вторичный contrast с direct;
- чувствительность к одному разговорному paraphrase;
- повторяемость бинарного результата при temperature = 0;
- адаптивный бюджет 6 или 10 вызовов.

Почему upfront BVC полезен пользователю

- алгоритм включается осознанно до первой правки, когда цена неверного направления ещё не превратилась в цепочку изменений;
- на выходе пользователь получает один проверяемый план с причиной, стратегией, зависимостями и тестами, а не набор несвязанных мнений;
- решение об углублении анализа принимает прозрачный gate, а не неограниченный агентный диалог;
- бюджет ограничен 6–10 вызовами и может быть показан в интерфейсе вместе с фактическими токенами и стоимостью;
- разговорная формулировка проходит через тот же строгий контракт анализа, что удобно для vibe-coding-сценария.

BVC тем самым добавляет перед генерацией кода компактный слой инженерного решения: пользователь сохраняет контроль, а система делает основания плана видимыми.

2. Upfront BVC как рабочий контракт

2.1. Когда пользователь включает алгоритм

Upfront BVC уместен не для каждой автодополняемой строки. Он предназначен для момента, когда пользователь ещё не разрешил менять код и хочет повысить качество первого инженерного решения. Типовые входы:

- необходимо исправить дефект с неочевидной причиной;
- предполагается изменение нескольких модулей или API-контракта;
- пользователь уже запросил план у нескольких агентов и хочет синтезировать их позиции;
- нужно заранее сформулировать тесты и регрессионные инварианты;
- разговорная постановка неоднозначна, но фактическое требование можно проверить по репозиторию.

Активация является явной. Это сохраняет контроль пользователя над ценой: режим direct остаётся доступным для дешёвых и очевидных правок, а BVC выбирается там, где ошибка первоначального плана

потенциально дороже нескольких дополнительных вызовов.

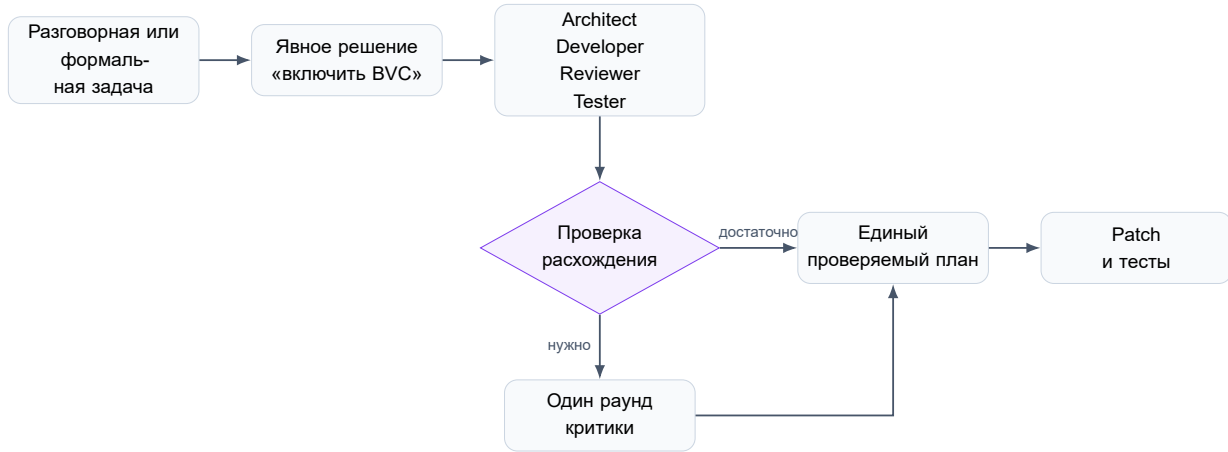


Рис. 1: Контракт upfront BVC: анализ отделён от редактирования кода.

2.2. Разделение ответственности

Четыре роли не голосуют за свободный текст целиком. Они формируют структурированные решения:

1. **Architect**: локализация причины, границы и зависимости;
2. **Developer**: конкретная стратегия изменения;
3. **Reviewer**: совместимость, инварианты и нежелательные эффекты;
4. **Tester**: наблюдаемое поведение и достаточные проверки.

После независимого раунда gate измеряет расхождение D_{vote} и неполноту D_{cov} . Если требуется критика, каждая роль видит структурированные позиции остальных и уточняет своё решение. Синтезатор выдаёт один план, а patch-генератор получает уже согласованную спецификацию.

2.3. Исполняемая последовательность одной сессии

Фраза «включить алгоритм заранее» в эксперименте означает конкретный порядок, а не общий совет писать более длинный prompt:

1. пользователь фиксирует issue либо уже составленный набором агентов черновой план и включает BVC до первой правки;
2. система прикрепляет одинаковый retrieval-контекст репозитория и запускает четыре независимых role-вызова;
3. JSON каждого ответа валидируется по четырём обязательным осям; отсутствующие или некорректные поля не угадываются, а входят в D_{cov} ;
4. по множествам файлов, зависимостей и тестов вычисляется расстояние Жаккара, по стратегии — расстояние замороженных категорий; среднее попарное расстояние образует D_{vote} ;
5. при $D_{\text{vote}} > 0,30$, $D_{\text{cov}} > 0,25$ либо отсутствии оси с двумя валидными голосами выполняется ровно один critique-раунд;
6. синтезатор формирует единый план, после чего отдельный вызов создаёт unified diff; только затем запускаются инструменты и тесты.

Число успешных модельных вызовов поэтому известно формулой

$$C_{\text{BVC}} = 4 + 4I_{\text{crit}} + 1 + 1 = 6 + 4I_{\text{crit}} \in \{6, 10\}.$$

Пользователь заранее видит верхнюю границу 10, но оплачивает четыре critique-вызова только при срабатывании gate.

2.4. Почему это не скрытый chain-of-thought

Пользователю не требуется показывать необработанный внутренний ход рассуждения. Полезный артефакт — структурированная карточка решения: файлы и символы, класс изменения, зависимости, инварианты, тесты, решение gate и фактический бюджет. Эти поля можно проверить по репозиторию, сохранить в журнале и передать инструментальному агенту.

3. Экспериментальный дизайн

3.1. Основная серия

Использована замороженная выборка из 60 задач SWE-bench Verified [2, 4]: 30 medium, 27 hard и 3 very hard, всего 11 репозитория. Для каждой задачи все методы видели одинаковое issue и одинаковый набор исходных файлов. Модель, temperature = 0, лимит patch и официальный evaluator также совпадали.

Выборка была outcome-blind: взяты все 3 задачи > 4 часов, 27 из 42 задач на 1–4 часа и 30 из 261 задач на 15 минут–1 час. Внутри репозиторийных квот порядок определял SHA-256 от фиксированного seed и instance_id; difficulty не входила в prompt. Общий BM25 context [6] содержал 2–8 файлов и 71 369–111 192 символа на задачу. Все 245 файлов совпали с raw-содержимым нужных base commit. Gold patch, test patch и скрытые тестовые списки generation runner не читал.

Таблица 1: Методы и стоимость их управляющей структуры.

Метод	Что происходит до patch	Вызовы	Основная роль в сравнении
Direct	ничего	1	отсутствие upfront-плана
Один план	один структурированный план	2	ценность одного планирования
BVC	4 роли, адаптивная критика, синтез	6/10	исследуемый workflow
Полный совет	те же роли, всегда критика	10	контроль полного бюджета

Использовалась Yandex Cloud AI Studio: requested-модель deepseek-v4-flash, returned URI gpt://deepseek-v4-flash/latest [8], prompt-версия main60-3.0.0, лимиты 4 096 output-токенов для роли и плана и 16 384 для patch. Порядок методов детерминированно вращался. Каждый prompt, ответ, usage, finish reason, длительность, plan и patch сохранялись с SHA-256. Успех определялся официальным контейнерным evaluator: diff должен применяться к base commit, а все обязательные fail-to-pass и pass-to-pass тесты — пройти. Пустые, неразбираемые и неприменимые patches не исправлялись вручную и не удалялись. Отдельный параметр thinking/non-thinking не передавался: использован provider default, а возвращённое поле reasoning_content сохранено. Provider seed недоступен, а алиас latest не гарантирует неизменность серверной ревизии при будущем повторе.

3.2. Парное определение эффекта

Для каждой задачи i фиксируются бинарные исходы $Y_{i,BVC}$ и $Y_{i,direct}$. Оценка:

$$\hat{\Delta}_{B-D} = \frac{1}{60} \sum_{i=1}^{60} (Y_{i,BVC} - Y_{i,direct}).$$

Интервал строится paired bootstrap с сохранением страт сложности, а точный тест использует только дискордантные пары [1, 3]. Такое определение отвечает на вопрос, какие именно одинаковые задачи решились по-разному.

3.3. Разговорная серия

На 15 заранее определённых задачах исходное требование было переформулировано моделью в разговорном стиле до получения результатов основной серии. Кодовый контекст, метод, модель и evaluator не изменялись. Сравнивались direct, один план и BVC. Один paraphrase позволяет проверить работоспособность интерфейсного сценария, но не весь спектр естественного языка.

Каждый из 15 model-generated paraphrase прошёл отдельную модельную проверку сохранения явно-го поведения, граничных условий, точных литералов и score, затем был перечитан вручную; использовался один принятый paraphrase на задачу. В итоговой metadata сохранилась косметическая метка prompt_variant=formal, но prompts были построены из отдельного файла разговорных входов и проверены по сохранённым API-артефактам. Регенерация только ради имени поля не выполнялась, чтобы не подменять outcome новой случайной реализацией.

3.4. Повторы

На тех же 15 задачах для трёх методов получено по три формальных outcome. Дополнительные 90 строк объединяются с 45 соответствующими строками основной матрицы. Измеряются число решённых задач в каждом повторе, доля задач с совпадением всех трёх исходов и среднее попарное согласие.

4. Первичный результат и вторичный описательный сигнал

BVC решил 5/60 задач, direct — 4/60. Попарная таблица:

Заранее заданным первичным contrast всей основной матрицы был BVC минус один план. Он дал 5/60 против 7/60: $-3,3$ п. п., интервал $[-10,0; +3,3]$, $p = 0,625$. Гипотеза преимущества не поддержана; неухудшаемость с margin -5 п. п. также не установлена, поскольку нижняя граница интервала равна -10 п. п.. Сравнение с direct ниже является заранее заданным вторичным ориентиром.

Таблица 2: BVC и direct на одинаковых 60 задачах.

	Direct: решено	Direct: не решено	Всего
BVC: решено	4	1	5
BVC: не решено	0	55	55
Всего	4	56	60

Точечная разность:

$$\hat{\Delta}_{B-D} = \frac{1 - 0}{60} = 0,0167 = +1,7 \text{ п. п.}$$

95% стратифицированный bootstrap-интервал равен $[0,0; +5,0]$ п. п.. Дискордантна только одна задача, поэтому точный двусторонний критерий Мак-Немара даёт $p = 1,0$. Следовательно, корректная формулировка результата: *получен положительный описательный сигнал качества относительно direct*; статистическое превосходство не доказано. При коррекции двух secondary-contrast методом Holm adjusted- p также равен 1,0. Нулевая нижняя граница bootstrap-интервала обусловлена структурой данных: нет ни одной задачи, решённой только direct, поэтому в любой resampling-реплике разность BVC–direct неотрицательна. Это свойство наблюдаемой матрицы, а не самостоятельное доказательство положительного эффекта.

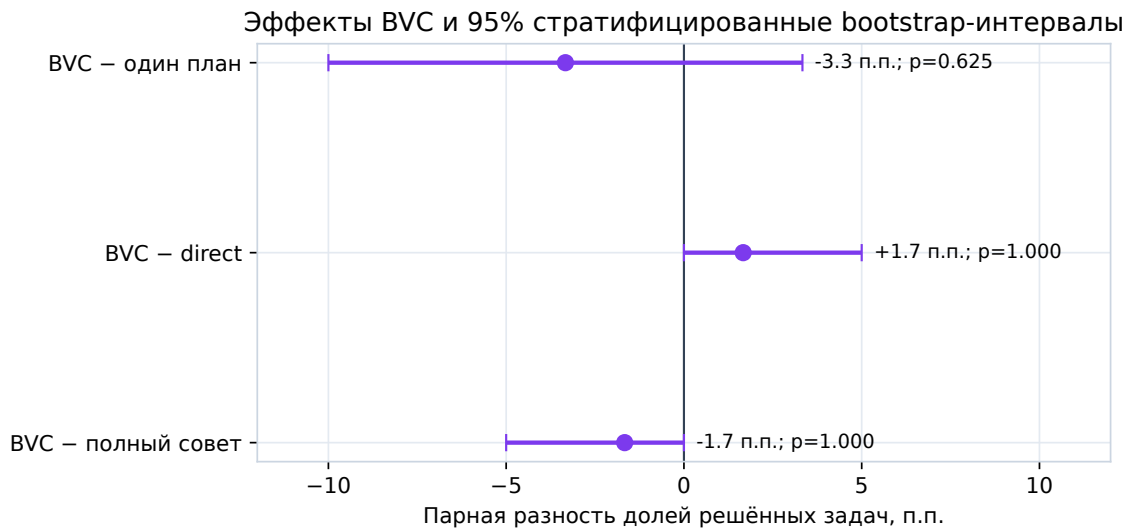
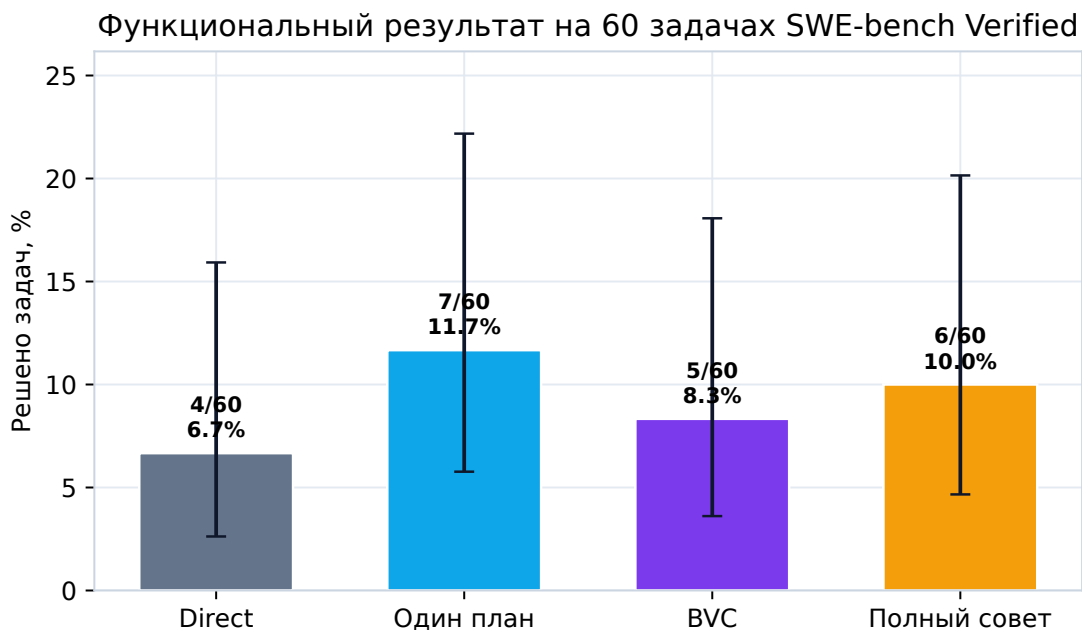


Рис. 2: Парные эффекты BVC относительно трёх заранее заданных baseline.

Полный совет решил 6/60, BVC — 5/60; пять успехов совпали. Результаты первичного contrast и полного совета не отменяют арифметику вторичного сигнала против direct, но точно ограничивают его интерпретацию: текущая выборка не подтверждает, что многоролевой upfront-анализ лучше простого структурированного плана.



Вертикальные интервалы: 95% Wilson CI для каждой доли; сравнение методов является парным.

Рис. 3: Функциональные outcomes и 95% Wilson-интервалы отдельных долей.

Wilson-интервал BVC равен [3,6; 18,1]%, direct — [2,6; 15,9]% [7]. Широкое перекрытие визуализирует неопределённость, которую нельзя скрывать одной разностью +1,7 п.п..

5. Разговорная постановка и vibe coding

5.1. Функциональный результат

Таблица 3: Официальные outcomes на 15 задачах при смене стиля запроса.

Метод	Формально	Разговорно	Разность	Согласие
Direct	0/15	2/15	+13,3 п. п.	13/15 (86,7%)
Один план	1/15	1/15	0,0 п. п.	15/15 (100,0%)
BVC	0/15	1/15	+6,7 п. п.	14/15 (93,3%)

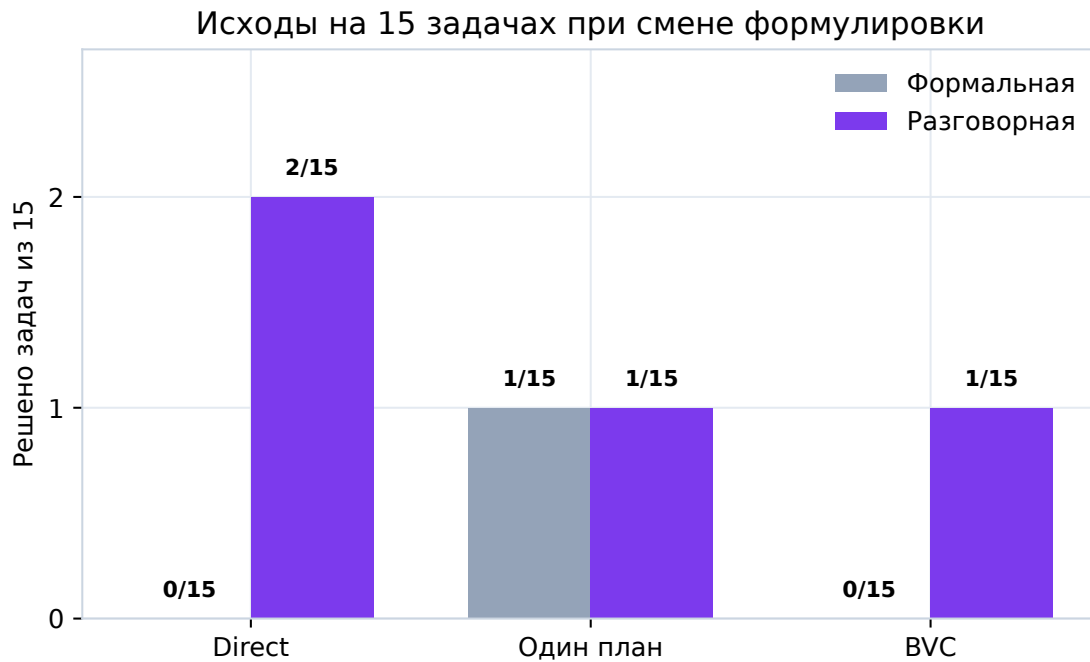


Рис. 4: Формальная и разговорная формулировки при неизменном evaluator.

В разговорной серии BVC и один план решили по одной задаче. Для BVC формальный и разговорный outcome совпал на 14/15 instances. Наблюдаемое изменение BVC равно +6,7 п. п., но при одной дискордантной задаче $p = 1,0$. Практически значим здесь не вывод о превосходстве языка, а то, что разговорный ввод прошёл полный структурированный pipeline: роли, gate, план, patch и официальные тесты. Разности в таблице не превышают диапазон 0–2 успеха из 15, наблюдавшийся между формальными повторами тех же методов; поэтому знак style-shift нельзя интерпретировать причинно.

5.2. Разность разностей

Для сравнения относительного style-shift BVC и одного плана вычислена разность разностей:

$$\hat{\delta} = (\hat{p}_{B,vibe} - \hat{p}_{B,formal}) - (\hat{p}_{S,vibe} - \hat{p}_{S,formal}) = +6,7 \text{ п. п.}$$

Стратифицированный bootstrap-интервал равен $[0,0; +20,0]$ п. п.. При малом $n = 15$ это только описательный post-hoc показатель: он не доказывает причинный эффект стиля и задаёт гипотезу для серии с несколькими независимо сгенерированными paraphrase. Нулевая нижняя граница здесь также следует из конструкции: outcomes одного плана совпали при обоих стилях, а у BVC нет задачи, решённой только в формальной серии.

6. Повторяемость при temperature = 0

Для BVC числа решений по трём формальным запускам равны 0, 2 и 0 из 15. Все три outcome совпали на 13/15 задачах; среднее попарное согласие составило 91,1%. Те же значения согласия получены для direct и одного плана, хотя распределение успехов различалось.

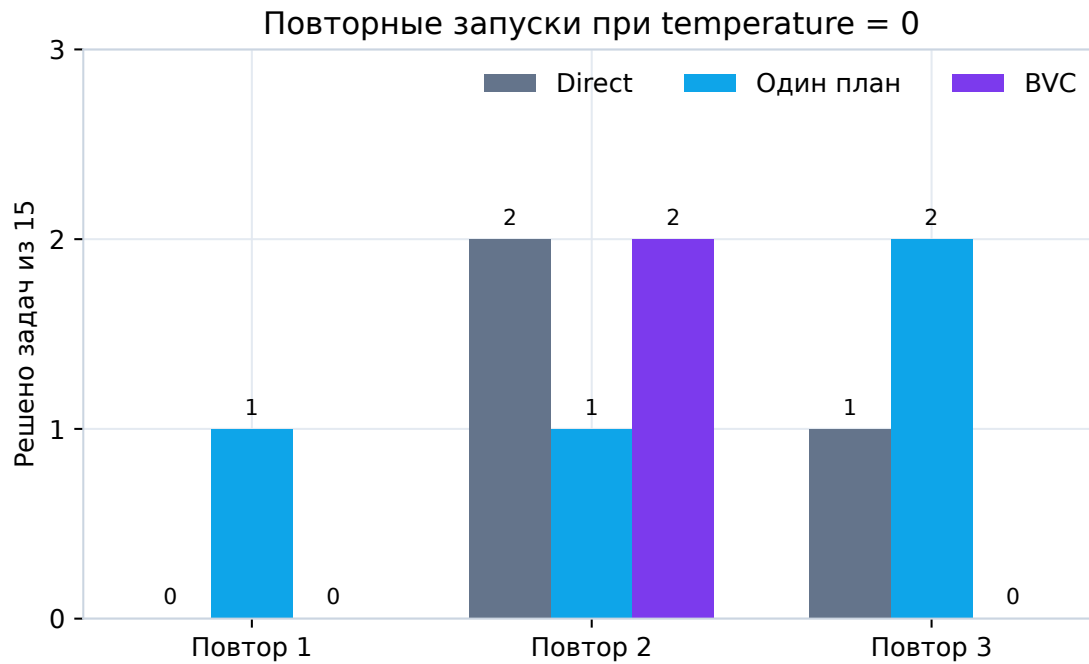


Рис. 5: Вариация числа успешных patch между тремя повторами.

Для продуктового интерфейса этот результат означает, что $\text{temperature} = 0$ нельзя показывать как гарантию идентичного outcome. Попарное согласие 91,1% можно отображать только как наблюдаемую audit-метрику данной 15-задачной серии, а не как калиброванную уверенность. Слой не оценивает дисперсию 60-задачного contrast напрямую, но колебание 0–2 успеха предупреждает, что различие в одну–две задачи не является устойчивым доказательством качества.

7. Адаптивный бюджет upfront-анализа

Полный многоролевой маршрут требует 10 вызовов. BVC завершил 32/60 задач после 6 вызовов и выполнил критику для 28/60. Среднее число вызовов:

$$\bar{C} = \frac{32 \cdot 6 + 28 \cdot 10}{60} = 7,8667.$$

Относительно обязательного полного совета:

$$1 - \frac{7,8667}{10} = 21,3\%.$$

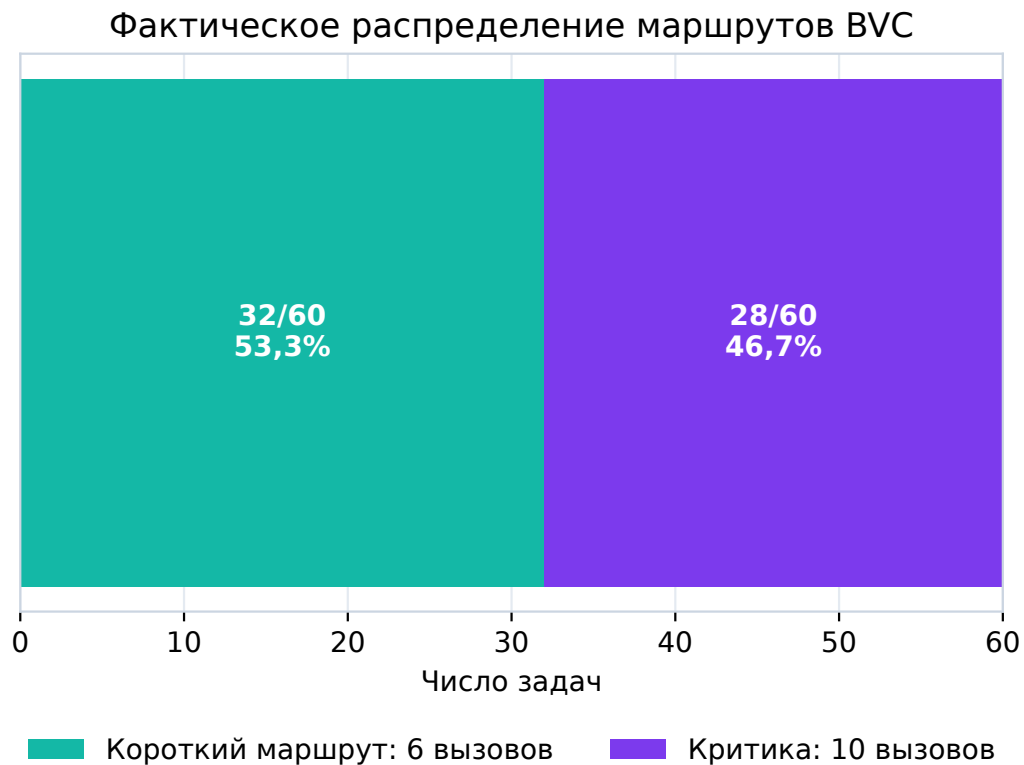


Рис. 6: Маршрутизация BVC между стандартным и расширенным режимом.

Таблица 4: Экономика BVC относительно полного совета.

Метрика	Полный совет	BVC	Снижение
Вызовы	600	472	21.3%
Токены	18 365 172	14 282 809	22.2%
Стоимость, руб.	2 402,61	1 838,46	23.5%
Стоимость на задачу, руб.	40,04	30,64	23.5%

Адаптивность особенно важна для vibe coding: человек может запросить многоролевую проверку, не обязуясь оплачивать максимальный раунд на каждой задаче. Диапазон 6–10 известен заранее, а фактический маршрут объясняется числовыми диагностическими признаками.

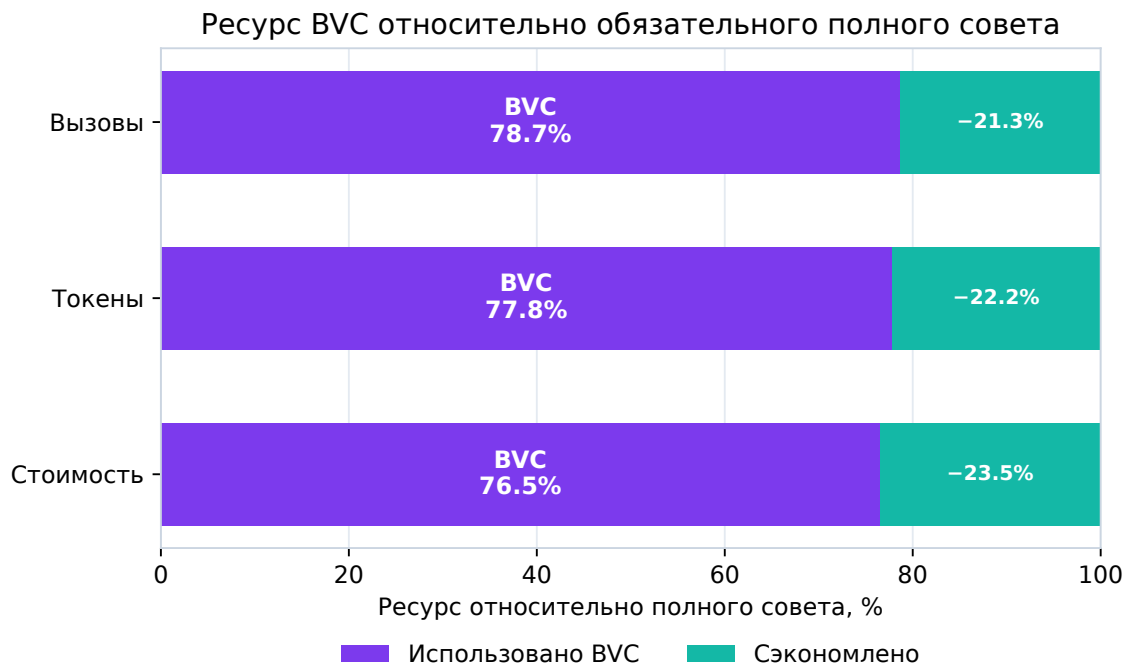
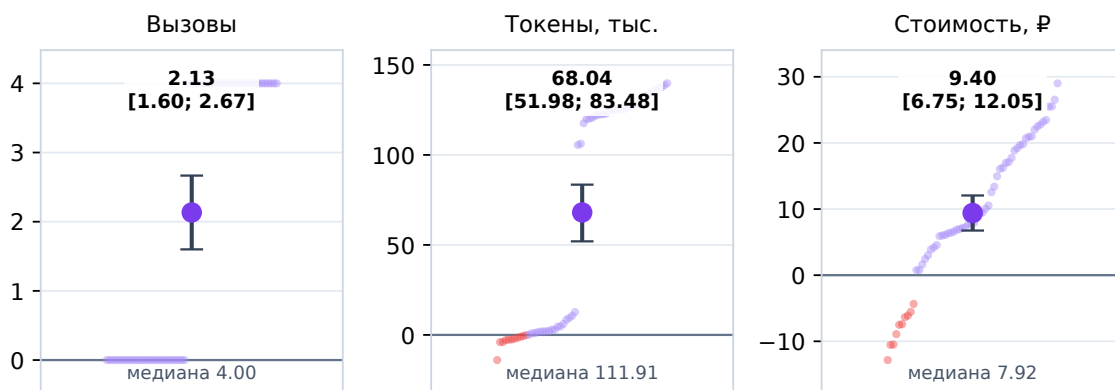


Рис. 7: Фактический ресурс BVC как доля ресурса полного совета.

Агрегаты дополнительно проверены на уровне одинаковых задач. Для $\Delta X_i = X_{i,\text{fixed}} - X_{i,\text{BVC}}$ средняя парная экономия равна 2,13 вызова, 68 039 токенов и 9,40 руб. на задачу. Соответствующие 95% bootstrap-интервалы равны [1,60; 2,67] вызова, [51 980; 83 480] токенов и [6,75; 12,05] руб. Bootstrap выполнялся 10 000 раз внутри medium/hard/very-hard страт; положительное значение означает экономию BVC. При этом на уровне отдельных задач BVC использовал больше токенов в 11/60 случаях и имел большую расчётную стоимость в 10/60; агрегатная экономия не означает экономию на каждой задаче.

Средняя парная экономия на задачу: полный совет – BVC



Малые точки: 60 пар; крупная точка: среднее; линия: 95% стратифицированный bootstrap-интервал.

Рис. 8: Парная экономия на задачу и 95% стратифицированные bootstrap-интервалы среднего.

8. Интеграция в Хунапсе

8.1. Команда и момент подтверждения

Дипломная архитектура [10] может использовать явную команду /bvc или переключатель «Проверить план до изменения кода». До запуска интерфейс показывает верхнюю границу 10 вызовов; после первого

раунда — выбранный маршрут. Отдельное подтверждение не требуется, если пользователь уже включил режим и бюджет находится в заявленном диапазоне.

8.2. Карточка плана

Результат синтеза должен отображать:

- root cause: файлы, символы и наблюдаемая причина;
- fix strategy: класс и шаги изменения;
- dependencies: контракты, которые необходимо обновить или сохранить;
- test coverage: существующие и новые проверки;
- D_{vote} , D_{cov} , причины gate;
- фактические вызовы, токены и оценочную цену.

Такая карточка является контрактом между предварительным анализом и инструментальным агентом. Если агент отклоняется от списка файлов или инвариантов, интерфейс может запросить новое решение, а не молча расширять patch.

8.3. Переход к инструментальному циклу

После BVC начинается обычный цикл поиска, редактирования и тестирования:

план → правка → тесты → наблюдение → уточнение.

BVC не заменяет инструменты. Его проектная задача — структурировать вход в этот цикл: заранее предложить, что менять и как доказать корректность; данный эксперимент не установил общего повышения качества. Результаты официального evaluator в исследовании выполняют ту же роль наблюдения, которую в IDE исполняют локальные тесты.

8.4. Когда достаточно короткого режима

Если роли независимо сходятся по локализации, стратегии и тестам, повторное обсуждение не добавляется. Если обнаружена неполнота или существенное расхождение, один раунд критики оплачивается автоматически. Наблюдаемая выборка дала 53,3% коротких маршрутов. Это конкретная основа для UI: индикатор бюджета может показывать не абстрактную «сложность», а фактическую долю случаев, когда дополнительная критика понадобилась.

9. Практические расчёты эффективности

9.1. Стоимость одного запуска

Средняя расчётная стоимость BVC равна 30,64 руб. на задачу. Короткий маршрут в наблюдаемой выборке стоил в среднем 22,78 руб., расширенный — 39,62 руб. Разность 16,84 руб. является описательным различием между группами задач, выбранными gate. Она смешивает цену четырёх critique-вызовов с различиями самих задач и поэтому не является причинной оценкой цены критики.

9.2. Стоимость успешного результата

BVC израсходовал 1 838,46 руб. и решил пять задач:

$$\frac{1\,838,46}{5} = 367,69 \text{ руб./решённую задачу.}$$

Полный совет:

$$\frac{2\,402,61}{6} = 400,44 \text{ руб./решённую задачу.}$$

Описательная экономия BVC по этой нормировке равна 8,2%. Из-за малого знаменателя показатель не доказывает общего преимущества, но подтверждает, что сокращение бюджета в данной серии не было куплено ростом наблюдаемой цены успеха относительно полного совета.

9.3. Цена дополнительной проверки

Сравнение с direct отвечает на вопрос качества, но цена режимов структурно различна: direct стоит в среднем 5,21 руб., BVC — 30,64 руб. Поэтому BVC следует включать не как постоянный заменитель любой генерации, а как осознанную предварительную проверку в задачах, где дополнительный анализ оправдан ценой ошибки. Эксперимент показывает измеримый функциональный сигнал $+1/60$, но не оценивает стоимость производственного дефекта; такое решение остаётся за пользователем или политикой IDE.

10. Научная и практическая значимость

Первый результат — операционализирован пользовательский сценарий «сначала обсудить, потом писать код»: момент включения, роли, структура вывода и граница бюджета заданы явно.

Второй результат — вторичный положительный task-paired сигнал относительно direct: одна дополнительная решённая задача без потерь на общих успехах. Его сила описана точно: $+1,7$ п. п., $p = 1,0$, без заявления статистического превосходства.

Третий результат — BVC прошёл полный pipeline при одной разговорной формулировке, дал 1/15 успех и 93,3% согласия outcome со своей формальной версией; это документирует сценарий, но не устанавливает общую устойчивость.

Четвёртый результат — стоимость многоролевого анализа стала управляемой: gate сократил токены на 22,2% и расчётную стоимость на 23,5% относительно обязательной критики.

В совокупности эти данные поддерживают дальнейшее развитие Хунапсе как IDE, в которой пользователь может выбирать не только модель, но и проверяемый режим принятия решения. Это и есть ключевая практическая польза алгоритма: он встраивает между намерением пользователя и правкой кода ограниченный, объяснимый этап проверки, глубина которого адаптируется по наблюдаемым диагностическим признакам. Ссылка на диплом [10] фиксирует происхождение архитектуры; числовая часть статьи основана на отдельном исполняемом эксперименте.

11. Границы интерпретации и следующий тест

Основная выборка ограничена 60 задачами, а разговорная — 15 задачами и одним созданным моделью paraphrase на задачу. Здесь «vibe coding» операционализирован только как однократная разговорная переформулировка требования до генерации; интерактивное программирование с последовательными уточнениями, инструментами и пользовательской обратной связью не исследовалось. Не включён equal-budget best-of- k или self-consistency baseline, поэтому работа не показывает, что многоролевая декомпозиция является лучшим использованием того же бюджета. Контекст ограничен BM25-выборкой, а patch создавался одним вызовом без тестовой обратной связи. Число функциональных успехов мало, поэтому интервалы широки. Следующий подтверждающий тест должен заранее зафиксировать новую выборку, несколько независимых разговорных формулировок, primary-contrast BVC-direct и достаточный объём для обнаружения эффекта порядка нескольких процентных пунктов. Текущие данные подходят для постановки такой гипотезы, но не для post-hoc заявления гарантированного улучшения. Кроме того, актуальный аудит SWE-bench Verified указывает на остаточные дефекты тестов и контаминацию, из-за чего benchmark больше не рекомендуется как мера frontier coding capability [5]. Здесь он используется как фиксированный одинаковый evaluator четырёх workflows, а не как абсолютный рейтинг модели.

Три статьи проекта анализируют одну основную матрицу 240 запусков, а не три независимые репликации: настоящая статья выделяет upfront-сценарий и чувствительность к стилю, две другие — адаптивный бюджет и аудируемость.

12. Заключение

Upfront BVC реализует конкретный и ценный пользовательский запрос: включить многоролевою проверку до исправления ошибки или до передачи плана агентам. В первичном сравнении он дал пять успешных patch против семи у одного плана; преимущество и неухудшаемость не установлены. Во вторичном сравнении пять успехов против четырёх у direct дают положительный описательный сигнал $+1,7$ п. п. при adjusted- $p = 1,0$. В разговорной серии BVC решил 1/15 и сохранил 93,3% согласия с формальным outcome.

Адаптивный gate использовал короткий маршрут в 32/60 задачах и сократил токены более чем на одну пятую относительно полного совета.

Практическая ценность BVC уже подтверждена там, где эксперимент имеет достаточно прямое измерение: ресурсная адаптивность показана напрямую, а улучшение качества пока является небольшим наблюдаемым сигналом, не статистическим доказательством. Именно в такой форме результаты экспериментально продолжают дипломный проект Хунапсе [10] и дают основу для следующей, заранее рассчитанной серии. Алгоритм ценен не обещанием заменить инженерное суждение, а тем, что делает предварительную проверку включаемой, ограниченной, объяснимой и измеримой.

Декларации

Финансирование и конфликт интересов отсутствуют. LLM использовались для генерации экспериментальных ответов, разговорных paraphrase и языковой редакции; функциональный outcome определён официальным исполняемым evaluator, а числовые значения вычислены из сохранённых JSON/JSONL. Диплом [10] является источником архитектуры Хунапсе/BVC, но не источником экспериментальных результатов статьи.

Литература

- [1] B. Efron и R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1993. doi: [10.1201/9780429246593](https://doi.org/10.1201/9780429246593).
- [2] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press и K. Narasimhan. «SWE-bench: Can Language Models Resolve Real-World GitHub Issues?» В: *International Conference on Learning Representations*. 2024. URL: <https://arxiv.org/abs/2310.06770>.
- [3] Q. McNemar. «Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages». В: *Psychometrika* 12.2 (1947), с. 153—157. doi: [10.1007/BF02295996](https://doi.org/10.1007/BF02295996).
- [4] OpenAI. *Introducing SWE-bench Verified*. 13 авг. 2024. URL: <https://openai.com/index/introducing-swe-bench-verified/> (дата обр. 28.07.2026).
- [5] OpenAI. *Why SWE-bench Verified No Longer Measures Frontier Coding Capabilities*. 23 февр. 2026. URL: <https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/> (дата обр. 28.07.2026).
- [6] S. Robertson и H. Zaragoza. «The Probabilistic Relevance Framework: BM25 and Beyond». В: *Foundations and Trends in Information Retrieval* 3.4 (2009), с. 333—389. doi: [10.1561/15000000019](https://doi.org/10.1561/15000000019).
- [7] E. B. Wilson. «Probable Inference, the Law of Succession, and Statistical Inference». В: *Journal of the American Statistical Association* 22.158 (1927), с. 209—212. doi: [10.1080/01621459.1927.10502953](https://doi.org/10.1080/01621459.1927.10502953).
- [8] Yandex Cloud. *Миллион токенов контекста: запускаем DeepSeek V4 Flash в Yandex AI Studio*. 28 мая 2026. URL: <https://yandex.cloud/ru/blog/yandex-ai-studio-deepseek-v4-flash> (дата обр. 03.08.2026).
- [9] Д. Э. Халилов. *Хунапсе: интеллектуальная среда разработки с интегрированным AI-ассистентом*. GitHub. 2026. URL: <https://github.com/jabrailkhalil/xynapse> (дата обр. 03.08.2026).
- [10] Д. Э. Халилов. «Интеллектуальная среда разработки с интегрированным AI-ассистентом для российского рынка». Выпускная квалификационная работа. Москва: Национальный исследовательский университет «Высшая школа экономики», 2026.